A

30 cross-system jobs I run every week

> **Note**
>
> *Thirty jobs my staff does every week, organized as a working operator's catalog. Each entry names what crosses what, the artifact it produces, the prompt grammar that runs it, and the chapter that taught the pattern. Strip the agent names if your staff has different ones; the shape is what matters.*

The first edition of this book had a beginner prompt library organized by surface task: Email, Writing, Files, Data. Most of those prompts have been folded into the chapters where they make sense. This appendix is different.

These are the *jobs* a graduate of this book actually runs after the staff is built. Each one crosses two or more systems: Gmail and Notion, the filesystem and a scheduled job, an MCP and a named subagent, the browser and the journal. None of them are chat-shaped. None of them would work in a window where you can only paste text. That's the line between the prompts that lived in v1 and the jobs that live here.

Thirty jobs. Four categories. Each entry has the same shape: **what crosses what** (the cross-system ingredients), **the artifact** (where the result lands as a file), **the prompt grammar** (one representative phrasing), and **taught in** (the chapter that introduced the pattern).

Daily-driver jobs (8)

The eight jobs I run from a fresh `claude` session most mornings. They run in roughly this order on Tuesdays.

1: Morning Iris check *What crosses what:* Gmail MCP + Notion (Iris's drafts pages) + the named subagent `iris`. *Artifact:* Iris's draft queue summary, written to `~/work/journal/iris-{date}.md`. *Prompt grammar: "Show me what Iris landed overnight: what's in 'sent,' what's still in 'draft awaiting my review,' what got escalated. For each draft, one sentence on why she didn't auto-send. Don't open the inbox yet, just the queue." Taught in: Chapter 11.*

2: Newsroom standup *What crosses what:* The four newsroom agents (editor, researcher, fact-checker, headline) + their shared task file in `~/work/newsroom/board.md`. *Artifact:* A status table appended to `~/work/journal/`

`standup-{date}.md`. *Prompt grammar: "What is each newsroom agent working on right now? For each one: current task, last completed artifact, anything blocking. If two agents are working on the same input, flag the duplicate." Taught in: Chapter 18*.

3: Reuben's reconciliation *What crosses what:* The bookkeeper subagent (Reuben) + bank-CSV folder + Stripe MCP (read-only) + Notion finance database. *Artifact:* P&L delta vs. last week, filed to `~/work/finance/weekly/{date}.md`. Flagged transactions appended to a Notion page. *Prompt grammar: "Run Reuben. File the P&L delta vs. last week. Flag anything over $200 that doesn't match a known category, and anything that matches a category but sits more than two standard deviations from that category's last 12 weeks. Don't auto-categorize the flags: list them." Taught in:* Chapter 13.

4: Scheduled-job audit *What crosses what:* The scheduled-jobs registry + the log folder at `~/.claude/logs/jobs/` + the cron interpreter. *Artifact:* A status table written to `~/work/journal/jobs-{date}.md`, name, last-fired, next-fires, last exit code. *Prompt grammar: "List every scheduled job: name, when it last fired, when it next fires, last exit code. Anything that hasn't fired in its expected window, surface to the top. Anything that exited non-zero, show me the last 20 lines of its log." Taught in: Chapter 9*.

5: Weekly leads digest *What crosses what:* The Notion Leads database (*Chapter 22*'s landing page writes here) + the campaign-cost CSV at `~/work/data/ad-spend-{month}.csv` + the `your-tone.md` skill for the digest voice. *Artifact:* A one-page digest written to `~/work/marketing/leads-digest-{week}.md`, leads grouped by `utm_source` and `utm_campaign`, cost per lead, week-over-week deltas, top three converting CTAs. *Prompt grammar: "Pull last 7 days of rows from the Notion Leads DB. Group by utm_source then utm_campaign. Compute cost per lead from the campaign-cost CSV. Write the digest in my voice: lead with the deltas vs. last week, then the source breakdown, then the top three converting CTAs. File at `~/work/marketing/leads-digest-{week}.md`." Taught in: Chapter 22*.

6: Plan-mode dry run *What crosses what:* Plain-English task description + the file system Claude is about to touch + plan mode. *Artifact:* A numbered execution plan, on screen only, before any tool runs. *Prompt grammar: "Plan-mode the next thing on my todo: [TASK]. Don't execute. Walk it as a numbered list, each step, what it touches, what could go wrong. I'll approve or redirect before you start." Taught in: Chapter 10*.

7: Watchdog mesh status *What crosses what:* The four watchdog monitors + the orchestrator agent + their shared event log. *Artifact:* A 24-hour anomaly summary written to `~/work/journal/watchdog-{date}.md`. *Prompt grammar: "Ask the watchdog mesh what it noticed in the last 24 hours that I should care about. Filter: things that crossed a threshold, things that stopped firing, things that fired more than usual. Skip the routine." Taught in: Chapter 20*.

8: End-of-day shutdown *What crosses what:* SessionEnd hook + the journal folder + the next morning's scheduled brief inputs. *Artifact:* `~/work/journal/{date}-eod.md` plus an updated input list at `~/work/journal/_brief-inputs.md`. *Prompt grammar: "Summarize the day. File SessionEnd notes to `~/work/journal/`. Queue tomorrow's morning-brief inputs (any specific files or topics I want pulled). Tell me one thing I said earlier this week that I should follow up on next week." Taught in: Chapter 9*.

Agent-authoring jobs (10)

Jobs that *build* things. Most of these came out of a moment where I caught myself typing the same instruction for the third time and decided to file it as an artifact instead. Each produces a file in `.claude/`.

9: New subagent scaffold *What crosses what:* An existing agent file (Iris) read as a template + the new agent's permission scope + a path under `.claude/agents/`. *Artifact:* `.claude/agents/{NAME}.md`. *Prompt grammar: "Read* `.claude/agents/iris.md` *for the shape, then scaffold* `.claude/agents/[NAME].md` *for [ROLE]. Permissions: [SCOPE]. The agent should never [FORBIDDEN ACTION]. Return only the artifact, never the raw working data. Match Iris's structure." Taught in: Chapter 8.*

10: Refactor a drifting agent *What crosses what:* The agent's last 10 outputs (read as files) + the agent file + the original tone-of-voice skill. *Artifact:* A diff of the agent file, plus a one-paragraph diagnosis appended to the agent's changelog. *Prompt grammar: "Iris's drafts are getting too formal lately. Read the last 10 outputs in* `~/work/iris/sent/`. *Find the drift: specific patterns, not vibes. Propose a one-paragraph addition to her agent file that pulls her back. Don't write the addition yet; show me the diagnosis first." Taught in: Chapter 11.*

11: Author a new skill *What crosses what:* An existing skill file (the tone-of-voice skill) read as a template + the trigger condition + the skills folder. *Artifact:* `.claude/skills/{NAME}.md`. *Prompt grammar: "Write a skill at* `.claude/skills/[NAME].md` *for [TASK]. Read* `.claude/skills/your-tone.md` *for the shape. The trigger should fire when [SITUATION]. Keep it under 30 lines." Taught in: Chapter 17.*

12: Draft a hook from a wish *What crosses what:* The hook event menu + Slack MCP + the `settings.json` file. *Artifact:* A diff of `~/work/.claude/settings.json` plus the hook script under `~/work/scripts/`. *Prompt grammar: "Whenever Claude finishes a task that took longer than five minutes, ping Slack on* `#me`. *Pick the right hook event, draft the JSON for* `~/work/.claude/settings.json`, *explain in one sentence why you picked that event over the others. Don't apply yet: show me the diff." Taught in: Chapter 9.*

13: Draft an MCP tool *What crosses what:* An existing MCP server's source folder + the new tool spec + a smoke test. *Artifact:* A new tool function in `~/work/mcp-servers/personal-data/`, plus a test entry, plus one console run of the smoke test. *Prompt grammar: "Add a tool called [NAME] to my custom MCP at* `~/work/mcp-servers/personal-data/`. *It takes [INPUTS] and returns [OUTPUT]. Pattern-match the existing tools in the same server. Add a smoke test to the test file. Run it once and show me the output." Taught in: Chapter 16.*

14: Author the verify script *What crosses what:* The agent file + the watchdog's nightly runner + a known-good test input. *Artifact:* `run-and-verify.sh` next to the agent file. *Prompt grammar: "Write* `run-and-verify.sh` *for the agent in* `.claude/agents/[NAME].md`. *Minimum: a smoke test the watchdog can run nightly that exits 0 if the agent still works and non-zero with a useful message if it doesn't. Don't make it perfect: make it informative." Taught in: Chapter 20.*

15: Promote a one-off into a skill *What crosses what:* The session transcript + the pattern in the last three prompts + the skills folder. *Artifact:* `.claude/skills/{NAME}.md`. *Prompt grammar: "That thing we just did, turn it into a skill at* `.claude/skills/[NAME].md`. *The trigger is [SITUATION]. The instructions are exactly what you and I just worked through, written as steps, not paragraphs." Taught in: Chapter 17.*

16: A/B test winner readout *What crosses what:* PostHog events (the `headline_variant` super-property from *Chapter 22*'s A/B test) + the Notion Leads database (conversion rows) + a stats helper that computes confidence intervals. *Artifact:* A one-paragraph readout written to `~/work/marketing/ab-tests/{date}.md`, variant A vs. variant B conversion rate, sample size per variant, confidence interval on the delta, called-winner-or-keep-running verdict. *Prompt grammar: "Read PostHog events for the last 14 days, split by* `headline_variant` *super-property. Join against the Notion Leads DB to compute conversion rate per variant. Compute the 95% confidence interval on the delta. If both variants have at least 500 page views and the CI excludes zero, declare a winner. Otherwise, keep running. Write the one-paragraph readout." Taught in: Chapter 22.*

17: Build a watchdog for an agent *What crosses what:* The agent's run-and-verify script + the watchdog mesh's monitor folder + a scheduled job. *Artifact:* A new monitor under `.claude/agents/watchdog/`, plus a cron entry, plus a Slack ping configured. *Prompt grammar: "Build me a watchdog that pings me if [AGENT] hasn't run in 48h, or if its last run had a non-zero exit, or if its output is empty when it shouldn't be. Use the same pattern as the bookkeeper-watchdog in* `.claude/agents/`*. Cron: daily at 6 PM." Taught in: Chapter 20.*

18: Re-prompt with a counterexample *What crosses what:* The wrong output (pasted) + the right output (pasted) + the agent file. *Artifact:* A diff of the agent file with the counterexample inlined. *Prompt grammar: "Here's an output the agent got wrong: [PASTE]. Here's what I wanted instead: [PASTE]. Update the agent prompt so it gets this case right next time. Add the counterexample inline in the agent file, not as a separate doc." Taught in: Chapter 8.*

Debugging jobs (6)

The Karpathy rule sits underneath all six: *paste what you saw, ask what broke.* The numbered fixes in *Appendix E* are the scaffolding for when paste-and-ask isn't enough. Each job here crosses logs + filesystem + a running process.

19: Paste-the-error (universal) *What crosses what:* The full error text + the file the error names + a diagnostic prompt. *Artifact:* A diagnosis on screen plus, if the fix is small, a diff of the named file. *Prompt grammar:"[PASTE FULL ERROR, every line, including the last]. What broke and what's the smallest fix? Don't rewrite the script: diagnose first, then propose the minimum change." Taught in: Appendix E.*

20: Hook hang diagnosis *What crosses what:* The hook script + the hook event log + a direct shell run of the same command. *Artifact:* A trace showing where the command blocks, plus a proposed timeout to add to the hook JSON. *Prompt grammar: "My SessionStart hook is hanging. Here's the hook command: [PASTE]. Test it directly outside the hook context. Tell me where it blocks, and propose a timeout I can add without breaking the hook's job." Taught in: Appendix E.*

21: MCP returning nulls *What crosses what:* The MCP server log + a smoke test of one tool + the keychain auth entry. *Artifact:* A diagnosis naming whether it's auth expiry, schema drift, or upstream API change, plus the smoke-test output. *Prompt grammar: "My [MCP] is returning null where it used to return data. Read the server config at [PATH]. Run a smoke test against one known-good query. Tell me whether it's auth expiry, schema drift, or upstream API change. Show me the smoke-test output." Taught in: Appendix E.*

22: Subagent loop *What crosses what:* The agent file + the last three session transcripts + the trigger that fired the loop. *Artifact:* A diagnosis identifying the loop trigger, plus a proposed stopping condition. *Prompt grammar:*

"Iris invoked herself 12 times in a row before I killed it. Read the agent file at `.claude/agents/iris.md` *and the last session log. Find the loop trigger and propose a stopping condition. Don't rewrite the agent yet: diagnose first." Taught in: Appendix E.*

23: Schedule didn't fire *What crosses what:* The job state file + the cron interpretation + the system wake schedule + the prior exit code. *Artifact:* A one-line diagnosis naming which of four causes (paused, asleep, prior-exit-disabled, cron-misread) explains the miss. *Prompt grammar: "Scheduled job [ID] should have fired at 6:30 AM. It didn't. Check job state, last exit code, the cron interpretation, and whether the machine was awake at fire time. Tell me which of those four is the cause."Taught in: Appendix E.*

24: Permission denied unexpectedly *What crosses what:* `.claude/settings.json` + `.claude/settings.local.json` + the action Claude tried. *Artifact:* The matching rule named, plus a proposed minimum-scope change to allow the case. *Prompt grammar: "Claude refused to write a file in* `~/work/`*. Read* `.claude/settings.json` *and* `.claude/settings.local.json`*. Tell me which rule blocked the write, and propose the smallest change to allow this case without opening the door wider." Taught in: Appendix E.*

"When something feels off" jobs (6)

The soft-signal set. These catch drift before it becomes incident. I run most of these during the Sunday governance hour from Chapter 29.

25: Cost trend check *What crosses what:* 30 days of `/cost` output by agent + a baseline from the prior 30 days + Appendix B's "what blows up your bill" list. *Artifact:* A trend table written to `~/work/journal/cost-{week}.md` with anomalies flagged. *Prompt grammar: "Pull the last 30 days of token spend by agent. Anything bending up that wasn't bending up last week? For each anomaly, hypothesis on cause. Cross-reference Appendix B's 'what blows up your bill' list before you guess." Taught in: Chapter 29.*

26: Conversion-rate trend audit *What crosses what:* The Notion Leads database (last 7 days vs. trailing 4-week average) + the dashboard's leads-by-source tile from Chapter 24 + the watchdog-mesh's funnel-anomaly source from *Chapter 20*. *Artifact:* A trend table written to `~/work/marketing/conversion-trend-{week}.md`, this week's rate by source, trailing 4-week average by source, and a tier-3 escalation flag if any source dropped more than 50% week-over-week. *Prompt grammar: "Compute this week's conversion rate by source from the Notion Leads DB. Compare to the trailing 4-week average. For any source where this week is more than 50% below average, flag a tier-3 escalation and propose a Slack ping for the watchdog mesh. File the table." Taught in: Chapter 22.*

27: Quiet-agent audit *What crosses what:* The scheduled-jobs log + each agent's last-run timestamp + their last output's content. *Artifact:* A list grouped into "trigger never fired" vs. "agent broke quietly," written to `~/work/journal/quiet-{week}.md`. *Prompt grammar:"Which agents haven't fired in the last seven days? For each one, tell me whether the trigger never fired (no input matched) or the agent broke quietly (errored or returned empty). Don't lump them." Taught in: Chapter 20.*

28: Slowness diagnosis *What crosses what:* Hook overhead measurements + MCP latency samples + `CLAUDE.md` size + session timing. *Artifact:* Numbers showing the actual bottleneck, plus a proposed fix. *Prompt grammar: "My sessions feel slower this week. Find the actual bottleneck: hook overhead, MCP latency,* `CLAUDE.md` *size, or something else. Measure, don't guess. Show me the numbers you found, then propose the fix." Taught in: Chapter 25.*

29: Promotion-readiness check *What crosses what:* The agent's 30-day correction log + the proposed new permission + a reversal mechanism. *Artifact:* A go/no-go recommendation plus a diff of the agent file (and `settings.json` if the permission scope changes). *Prompt grammar: "Iris has been drafting for 30 days with zero corrections from me on the [CATEGORY] inbox. Should I promote her to send-without-asking on that category? What would change about the agent file? What's the smallest reversal mechanism if she botches one?" Taught in: Chapter 11.*

30: Garden weeding *What crosses what:* Every file in `.claude/` + their last-modified times + their reference graph (which agent calls which skill). *Artifact:* An audit grouped into never-used / used-once / duplicates / conflicts, written to `~/work/journal/garden-{quarter}.md`. No changes applied. *Prompt grammar: "Read every file in `.claude/`. Tell me what's stale, redundant, or contradictory. Don't change anything yet, just the audit. Group by: never-used, used-once, duplicates-of-something-else, conflicts-with-something-else." Taught in: Chapter 29.*

Notes on using this appendix

The 30 count is a promise, not a coincidence. If you find yourself wishing job 31 existed, the answer is probably to refactor one of these, not to add another. The list got long the way a good knife collection gets long: by dropping the duplicates rather than by buying more. (Three jobs in this edition are swap-ins from earlier prints: the launch funnel from *Chapter 22* generated enough recurring work that *Weekly leads digest*, *A/B test winner readout*, and *Conversion-rate trend audit* earned their slots. The three jobs they replaced (a `.claude/`-diff job that overlapped with garden weeding, a slash-command-to-hook demotion that was a one-time refactor masquerading as a recurring job, and a soft-drift detector that overlapped with the quiet-agent audit) moved off the list. The 30 number is steady; the contents shift as the staff's work shifts.)

Each job assumes you're inside the `~/work/` folder and a `claude` session. The placeholder convention (`[ALL-CAPS-WITH-DASHES]`) carries over from the first edition; readers who used the v1 prompt library will recognize it.

When a prompt isn't working, the meta-move is the same as *Chapter 8*'s: paste the unhelpful output back, say *"this didn't help, what would have helped,"* and let Claude refine the prompt with you. Several of these thirty got their final shape that way.

The four categories map back to four chapters' worth of pattern: daily-drivers to *Chapter 9*, agent-authoring to *Chapter 8* / *Chapter 16* / *Chapter 17*, debugging to *Appendix E*, "feels off" to *Chapter 20* and *Chapter 29*. If a job here surprises you, walk back to its taught-in chapter: that's where the surrounding shape lives.